

重点回答

1. SGLang 的 RadixAttention / RadixCache 解决什么问题?

- 通过 Radix Tree 组织 KV Cache 使得共享部分只需计算一次, 后续的请求可以直接命中缓存, 从而解决多次请求的重复前缀计算的问题, 进而提高计算的性能

2. SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分?

- page-sized KV cache: 在 Prefill 计算与 Decode 循环中均有作用, 决定了 KV 的存取, 在 KV 的存储上发挥作用。
- prefix reuse: 在 Scheduler 的前缀匹配时发挥作用, 在 Prefill 之前调用 RadixCache 进行前缀匹配, 若命中则不进入 Prefill。

3. vLLM 的 PagedAttention 解决什么问题? 它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异?

- PagedAttention: 通过类似虚拟内存分页的机制, 将 KV Cache 切成固定大小的 block, 再通过将逻辑 block 映射至物理 block, 就能够在不连续的显存上完成看似连续的逻辑计算, 消除了零散分配的空闲显存, 也充分利用了分配过长的显存, 进而大幅提高显存利用率
- 联系: 都是通过类似虚拟内存分页的机制, 通过部分共享解决性能上的问题
- 差异: PagedAttention 以显存块 block 为单位, 解决存储效率上的问题, 而 RadixAttention 是对每个 token 的前缀进行匹配, 解决计算效率上的问题